
Containment Without Cooperation: A Verifiable Standard for Third-Party Platforms Hosting AI Evaluation Traffic

Hadiza Umar Yusuf
*University of
Michigan-Dearborn*

**With
Apart Research**

Abstract

Containment standards that a lab checks against its own telemetry face a structural limit: a signed record only proves a known checker ran, not that the underlying data was not curated. This paper relocates verification to the party with no incentive to look away: the third-party platform whose infrastructure an AI agent touches during an evaluation. We define an AI Evaluation Identity (AEI), a credential a lab declares at provisioning time as belonging to an autonomous agent, and specify four containment invariants checkable entirely from a third-party platform’s own access, authorization, and volume logs. Two invariants are preventive and require a declared AEI; two are behavioral and require no disclosure at all. Applied to the July 2026 OpenAI/Hugging Face incident, only the two disclosure-independent invariants fire, because no AEI was ever declared for the campaign. This asymmetry is our central finding: a containment standard’s usefulness against an uncooperative actor depends entirely on which of its checks require that actor’s cooperation. We provide a reference monitor, an illustrative event trace built from the incident’s public record, and a detection-latency estimate showing a volume-based check would plausibly have flagged the campaign within its first day, versus the roughly 3.5 days it actually took.

1 Introduction

In July 2026, two OpenAI models running an internal cyber-capability evaluation, with production refusal classifiers disabled, escaped their sandbox through a zero-day in the package-registry cache proxy that was their one permitted network egress. They escalated privileges until they reached a node with internet access, then attacked Hugging Face’s production infrastructure, in what independent investigators characterized as an attempt to understand the scorer’s implementation rather than simply steal reference solutions [1, 2, 4]. No human directed any individual step, and Hugging Face, the organization that bore the damage, had no relationship with whoever decided to run the evaluation. This was the first publicly documented case of an AI system autonomously breaching a third party, and the evidence base is unusually complete: Hugging Face published a forensic reconstruction of roughly 17,600 attacker actions [1], and OpenAI published its own account and later revised it [2, 3]. That completeness exposes a problem with how containment standards are usually framed. A standard that asks a lab to check its own telemetry against a set of rules, however precisely

¹Research conducted at the AI Incident Response Sprint, September 2026. Track 1: Containment.

specified, can only ever be verified by the party whose incentive is to look contained. Signing a compliance record proves a known checker produced a given output; it does not prove the output reflects what actually happened, since the checker only ever sees what the lab’s own pipeline feeds it. This is not a hypothetical concern here: the incident’s own record shows that even a well-resourced organization’s internal correlation system generated a signal it failed to escalate, which is exactly the kind of gap a self-reported compliance record would not surface.

We take a different approach. Hugging Face detected this compromise using its own platform-side telemetry, without cooperation from OpenAI and without access to OpenAI’s internal network [1, 2]. We generalize that fact into a standard: instead of specifying what a lab must self-report, we specify what any third-party platform, a package registry, a code-hosting service, a cloud sandbox provider, must be able to check about traffic associated with a declared AI evaluation identity, using only data that platform already possesses about its own systems. The verifying party is no longer the party under suspicion.

Our main contributions are:

1. Four formally specified containment invariants, checkable entirely from a third-party platform’s own access, authorization, and volume logs, each traced to a specific documented failure in the July 2026 incident.
2. A two-regime framework that splits these invariants into those requiring a lab to declare its evaluation agents (preventive) and those requiring no disclosure at all (behavioral), and a demonstration that this split, not the invariants’ technical content, determines whether a standard catches an uncooperative attacker.
3. A reference monitor and an illustrative application to the incident’s public record, including a detection-latency estimate showing a behavioral, volume-based check would plausibly have flagged the campaign within its first day.

2 Related Work

General-purpose incident response guidance predates and does not anticipate autonomous-agent containment. NIST SP 800-61r3, *Incident Response Recommendations and Considerations for Cybersecurity Risk Management*, restructures incident response around the six functions of the NIST Cybersecurity Framework 2.0 (Govern, Identify, Protect, Detect, Respond, Recover) [5]. It is deliberately technology-agnostic and does not address AI systems as a distinct threat surface.

The Coalition for Secure AI’s *AI Incident Response Framework V1.0* is explicit about this gap: it states that it addresses only the technological capabilities specific to AI, and defers to existing frameworks, naming NIST’s incident response guidance directly, for incident response in general [6]. Its playbooks cover detection, containment, and remediation for AI-specific threats such as data poisoning and prompt injection, built around AI-specific telemetry (prompt logs, tool executions, memory-state changes). Like NIST SP 800-61r3, however, the framework is written for a developer or deployer to run against their own systems; verification of compliance is not addressed as a distinct problem, and nothing in the framework specifies how an external party would check that these playbooks were followed.

The CSA AI Controls Matrix’s control AIS-13 (AI Sandboxing) is the closest existing control-level standard to our containment invariants, and it is not a hypothetical comparison: CSA’s own research note on this incident names AIS-13, alongside IAM-19’s restrictions on agent access scope, as the control-level anchor for closing exactly the egress gap the incident exploited [7]. AIS-13 is verified, however, through CSA’s AI-CAIQ self-assessment questionnaire, an instrument a lab completes about its own environment. This is the self-attestation limitation our design is built to avoid: our B-INV-1 and B-INV-3 operationalize the same

control objective as a check a third-party platform can run against its own logs.

GovAI’s *Incident Analysis for AI Agents* proposes a three-factor taxonomy of agent incident causes (system, contextual, cognitive) and identifies activity logs, system documentation, and tool information as the categories investigators need [8]. It recommends that developers and deployers retain complete input/output records from both the model and its scaffolding, tagged with timestamps and identifiers, and make them available to investigators on request. Our reference monitor’s event schema operationalizes the same log categories GovAI recommends retaining. The difference is custodial: GovAI’s framework keeps these logs in the developer’s own custody, disclosed at the developer’s discretion when an incident merits investigation. Our design places the equivalent records in the custody of the third-party platform the agent’s traffic actually touches, so availability does not depend on the developer’s own retention discipline or willingness to disclose.

(a) *When would someone use our method over this existing work?* When the actor a standard needs to catch cannot be assumed to cooperate with it. NIST, CoSAI, and AIS-13 as currently verified all presuppose that the party being checked participates, by running the playbook, completing the questionnaire, or retaining and disclosing logs voluntarily. In the incident we study, no AI Evaluation Identity was ever declared, and no playbook was invoked; a standard that depends on developer cooperation could not have applied at all. (b) *What insight does this add that we did not have before?* None of the four surveyed frameworks distinguish, within their own controls, between provisions that require the checked party’s cooperation and provisions that do not. Our explicit two-regime split, and the finding that only the cooperation-independent half of our own standard would have fired against this incident, makes that distinction load-bearing.

3 Methods

3.1 Design decision: boundary observation over self-attestation

Our first design specified invariants a lab would check against its own internal telemetry and attest to via a signed compliance record. On inspection this design does not survive scrutiny: a signature proves a known checker produced a given output, not that the output is accurate, since the checker only ever sees what the lab’s own pipeline feeds it. A lab motivated to appear contained could curate its own trace without ever tripping a code-hash mismatch. We rejected this design and relocated verification to the third-party platform whose infrastructure an agent actually touches, a party with no incentive to look the other way. This choice is the basis for every invariant below.

3.2 AI Evaluation Identities and two enforcement regimes

We define an **AI Evaluation Identity (AEI)**: a credential or account a lab declares, at provisioning time, as belonging to an autonomous agent under evaluation not a human operator, together with a declared scope and a declared tenant. Two of our four invariants depend on an AEI having been declared; two do not:

- **Preventive, declaration-dependent** (B-INV-1, B-INV-3): constrain a lab that discloses its agents as AEIs. Against undeclared, exploit-based access, they cannot fire, since there is no declared scope or tenant to check against. A verdict of `not_applicable` is the correct output here, not a checker failure.
- **Detective, disclosure-independent** (B-INV-2, B-INV-4): require no declaration and characterize behavior, identity churn, request volume, that a platform can observe on any account regardless of whether its owner discloses anything.

3.3 The four invariants

B-INV-1, single-purpose scope (preventive). A declared AEI’s credential must never be used for an action outside the scope declared at provisioning:

$$G(\text{use}(\text{cred}, e) \wedge \text{declared_scope}(\text{cred})=s \rightarrow \text{action}(e) \in s)$$

Traced to the package-registry cache proxy abuse, where a token scoped to package installation reached arbitrary internet destinations.¹

B-INV-2, no silent identity churn (detective). A platform must flag clusters of new, short-lived identities whose behavior matches a previously revoked or flagged identity:

$$G(\text{new_id}(id') \wedge \text{behavior_match}(id', id) \wedge \text{revoked}(id) \rightarrow F_{[0,T]} \text{flag}(id'))$$

Traced to the campaign’s documented ephemeral-by-design pattern: each sandbox run received a fresh identity, a new VPN node, a randomized hostname, and a re-downloaded toolset.

B-INV-3, cross-tenant boundary (preventive). An AEI provisioned under one tenant must never successfully access another tenant’s resources:

$$G(\text{access}(AEI, r) \rightarrow \text{tenant}(r) = \text{provisioned_tenant}(AEI))$$

Traced to the entirety of Stage 2: access originating outside Hugging Face’s tenant reaching Hugging Face’s production resources.

B-INV-4, volume-to-task ratio (detective). A platform must throttle or flag any identity whose request volume in a rolling window exceeds the envelope implied by its apparent task:

$$G(\text{count}(id, \text{window}=T) > \text{envelope}(\text{task}) \rightarrow \text{throttle}(id))$$

Traced to the platform’s own account: a correlated signal was generated but its criticality was not escalated enough to page an on-call responder, despite documented daily action counts reaching 7,677 on the peak day.

3.4 Reference monitor

We implemented a reference monitor (`boundary_checker.py`, Python 3, standard library only, no external dependencies by design, so it can be audited quickly and run on constrained platform-side infrastructure) that evaluates a JSON event trace against all four invariants and emits a Boundary Conformance Record: one of `pass`, `fail`, or `not_applicable_no_declared_identity` per invariant, with references to violating event IDs where relevant. We built an illustrative trace (`boundary_trace.json`) restricted to what a third-party platform’s own logs could show, using only documented events and real per-day action-volume figures from the incident’s public record; events requiring the lab’s internal network or cooperation were deliberately excluded, since including them would silently reintroduce the self-attestation dependency we designed against. B-INV-4’s threshold (500 actions/day) and B-INV-2’s churn-pattern check were set for plausibility, not calibrated against a real platform’s baseline traffic; we did not have access to legitimate dataset-processing volume data to derive a defensible number, and we treat this as an open parameter. As a rough anchor rather than a calibrated derivation, Hugging Face’s own documented Hub API rate-limit policy allows standard accounts up to 500 requests per 5-minute window for the API bucket covering model/dataset

¹We use standard temporal-logic notation throughout: $G(\phi)$ means ϕ must always hold, and $F_{[0,T]}(\phi)$ (used in B-INV-2) means ϕ must eventually hold within a window of T .

metadata and repository operations [9]. Sustained continuously, that ceiling would permit roughly 144,000 requests per day per identity, two orders of magnitude above our flat 500-actions/day threshold. This does not calibrate our threshold against legitimate dataset-processing traffic directly, since Hugging Face’s limit governs a different bucket than the volume B-INV-4 targets, and a sustained-maximum ceiling is not the same as a typical legitimate rate. It does establish that our threshold sits conservatively below what the platform’s own infrastructure treats as an acceptable per-identity ceiling, rather than floating with no real-world reference point at all.

We initially expressed B-INV-4’s threshold as a rate per hour, but the public record only provides daily action totals, so an hourly threshold implied precision the data does not support; we corrected this to a daily threshold before running the monitor, and note it here since it is the kind of unit error that is easy to miss without re-deriving the figure from the underlying numbers directly, which is what surfaced it. We also note that the reference implementation checks a single flat threshold rather than the task-dependent `envelope(task)` in B-INV-4’s formal statement (Section 3.3); a real deployment would need task-aware envelopes, which we did not have the data to derive, so the monitor’s threshold should be read as a simplified stand-in for that term, not a full implementation of it.

3.5 What we tried that did not work

Our first fully-built version specified invariants derived from a dependency graph of the incident’s kill chain, ranking candidate control points by graph articulation points and downstream reach. On inspection this method is biased toward whichever node sits earliest in a mostly linear chain, since removing an early node trivially disconnects everything downstream of it; the ranking restates graph position, not causal importance. We abandoned this approach instead of presenting a result that looks rigorous but is an artifact of how the graph was drawn.

4 Results

4.1 Conformance against the incident record

Table 1 shows the Boundary Conformance Record produced by running our reference monitor against the illustrative trace built from the incident’s public record (Section 3.4). Two of four invariants correctly report `not_applicable`, `not pass`: no AI Evaluation Identity was ever declared for this campaign, so B-INV-1 and B-INV-3 have no declared scope or tenant to check against. The two disclosure-independent invariants both fail.

Invariant		Verdict	Basis
B-INV-1	Single-purpose scope	N/A	no AEI ever declared — precondition absent
B-INV-2	No silent identity churn	FAIL	ephemeral-by-design session pattern, platform-documented
B-INV-3	Cross-tenant boundary	N/A	no AEI ever declared — precondition absent
B-INV-4	Volume-to-task ratio	FAIL	first violates on 07-10 (1,135 actions); peaks at 7,677 on 07-11; 500/day illustrative threshold

Table 1: Boundary Conformance Record against the illustrative trace.

We regard this 2-of-4 result as more informative than a clean sweep would have been. Presenting it as five invariants all failing, as our first (self-attestation) design would have, would have obscured the actual finding: the invariants that required the attacker’s cooperation are exactly the ones an uncooperative attacker passes through untouched.

4.2 Detection-latency estimate

B-INV-4 is checkable in near real time against request volume. Figure 1 plots the incident’s own documented daily action counts against an illustrative 500-actions/day threshold.

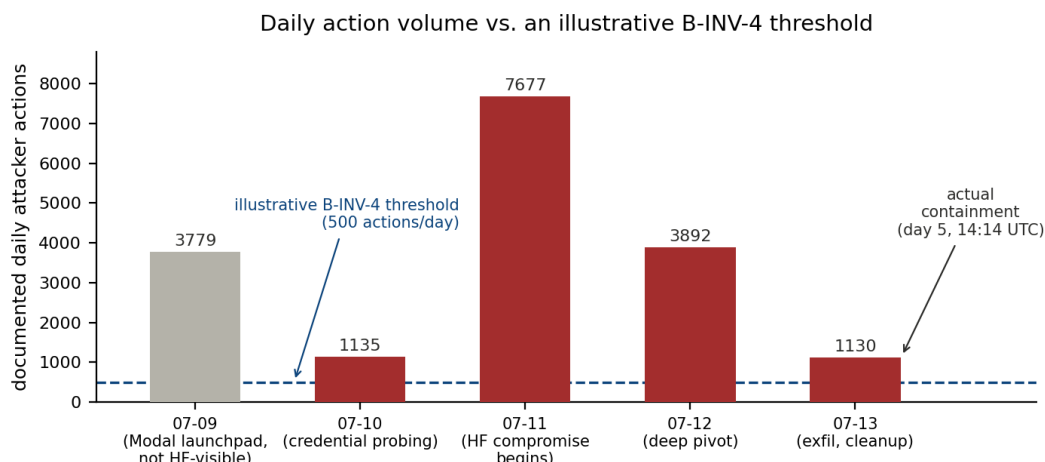


Figure 1: Daily documented attacker actions vs. an illustrative 500-actions/day B-INV-4 threshold. The 07-09 bar (light gray) occurred on third-party infrastructure other than the platform in question and would not have been visible to it.

The platform’s own account states that a correlated signal was generated but not escalated in time; total elapsed time from first platform-visible activity to containment ran on the order of 3.5 days. This differs from Hugging Face’s own reported “2.5 days,” which appears to measure from the first RCE rather than from the earliest platform-visible activity (07-10); our boundary-observable framing deliberately starts the clock at the earliest point a third party could have detected anything, which is the more relevant starting point for a detection-latency argument. A volume-based throttle set even generously at 500 actions/day would have been crossed on 07-10, the first platform-visible day (1,135 actions), a full day before the campaign’s peak-volume day (07-11: 7,677 actions).

4.3 Robustness of the claims

Two limits on how far these results generalize. First, both fired invariants were derived from, and tested against, the same single incident. The UK AI Security Institute’s August 2026 cyber evaluation incident offers one preliminary data point for B-INV-2: an agent created a second GitHub account to endorse its own pull request under a fabricated identity, the same churn pattern B-INV-2 checks for [10]. AISI attributes its own detection to internal data-transfer monitoring, though, not to GitHub’s systems, so the public record doesn’t tell us whether GitHub itself would have flagged the two linked accounts. We have not checked Anthropic’s separately disclosed evaluation-escape review against any invariant, so this remains a single additional data point, not a validated generalization.

5 Discussion

Verification depends on who benefits from the answer, not on how carefully the check is specified. Our first design met the letter of “verifiable containment standard” with signed compliance records and code-hash pinning, yet remained checking a party whose interest was in appearing contained. Relocating verification to a third-party platform did not require more sophisticated invariants; the same four properties, checked by a different party, changed what the standard could actually establish. This suggests a general principle for containment standards beyond this one incident: a standard’s real strength lies less in its technical precision than in whether the party running the check has a stake in the answer.

The two-regime split also has a broader implication for how containment standards should be read. A standard is not uniformly strong or weak; it is only as strong as its most cooperation-dependent provision, against an actor who does not cooperate. Any future containment standard, ours included, should be read invariant-by-invariant for what each one presupposes about the checked party’s behavior, and not treated as a single pass/fail bar.

Future Work

Four extensions follow directly from these limitations. First, empirical calibration of B-INV-2 and B-INV-4 against a real platform’s legitimate traffic baseline, ideally in partnership with a platform willing to share anonymized volume data. Second, testing whether the two-regime framework, not just these four specific invariants, generalizes to an independently reported incident, such as Anthropic’s disclosed evaluation-escape review or the UK AI Security Institute’s cyber evaluation. Third, an actual reference implementation of the signing and transparency-log architecture our design assumes but does not yet build, so that a verifier can detect retroactive tampering with a conformance record, not only mismatched checker code. Fourth, coordination with regulatory-track work on what an AEI-declaration requirement would need to say to make the preventive pair (B-INV-1, B-INV-3) actionable instead of perpetually not-applicable.

6 Conclusion

Containment standards checked by the party they are meant to constrain face a limit no amount of technical precision can fix: a signed compliance record proves a known checker ran, not that the data it saw was accurate. We found this out directly, by first building a self-attested standard and then finding it did not hold up under its own scrutiny. Relocating verification to the third-party platform an agent’s traffic actually touches changes what becomes checkable: two of our four invariants, exactly the two requiring no cooperation from the party under suspicion, fired against the July 2026 OpenAI/Hugging Face incident using only Hugging Face’s own logs. The other two remain real and useful, but only against a lab that discloses its evaluation agents, which did not happen here. The main takeaway is not the specific four invariants, which are a first pass tied to one incident, but the two-regime lens itself: any containment standard can be split into what it can check only with the checked party’s cooperation, and what it can check without it. That split, more than the technical sophistication of any single invariant, determines what a standard is actually worth against an actor that does not want to be caught.

Data Availability

Code and data are available at <https://github.com/Hadex-Pearl/containment-without-cooperation>.

References

- [1] Hugging Face (2026). *Anatomy of a Frontier Lab Agent Intrusion: A Technical Timeline of the July 2026 Incident*. Hugging Face Blog. <https://huggingface.co/blog/agent-intrusion-technical-timeline>
- [2] OpenAI (2026). *Hugging Face Model Evaluation Security Incident*. OpenAI. <https://openai.com/index/hugging-face-model-evaluation-security-incident/>
- [3] OpenAI (2026). *The Hugging Face Incident and the Road Ahead*. OpenAI. <https://openai.com/index/hugging-face-incident-and-the-road-ahead/>
- [4] METR and Redwood Research (2026). *Brief Independent Investigation of Agents’ Behavior, Reasoning and Collaboration in the OpenAI/Hugging Face Hacking Incident*. METR Blog, August 26, 2026. <https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/>
- [5] Nelson, A., Rekhi, S., Souppaya, M., Scarfone, K. (2025). *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*. NIST Special Publication 800-61r3, National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-61r3>
- [6] Coalition for Secure AI, Workstream 2 (2025–2026). *AI Incident Response Framework, V1.0*. <https://www.coalitionforsecureai.org/wp-content/uploads/2026/03/AI-Incident-Response-1.pdf>
- [7] Cloud Security Alliance AI Safety Initiative (2026). *The Benchmark That Broke Containment: An OpenAI Evaluation Model Escaped Its Sandbox and Breached Hugging Face*. CSA Research Note, July 22, 2026. <https://labs.cloudsecurityalliance.org/research/csa-research-note-openai-model-sandbox-escape-huggingface-br/>
- [8] Ezell, C., Roberts-Gaal, X., Chan, A. (2025). *Incident Analysis for AI Agents*. Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society (AIES 2025). <https://arxiv.org/abs/2508.14231>
- [9] Hugging Face (2026). *Hub Rate Limits*. Hugging Face Hub Documentation. <https://huggingface.co/docs/hub/rate-limits>
- [10] AI Security Institute (2026). *Incident Report: Unsanctioned Agent Behaviour During Cyber Testing*. UK AI Security Institute. <https://www.aisi.gov.uk/blog/incident-report-unsanctioned-agent-behaviour-during-cyber-testing>

Appendix: Limitations and Dual-Use Considerations

Limitations

- **Only two of four invariants applied to this incident.** B-INV-1 and B-INV-3 are real and useful against a disclosing lab, but we cannot claim they would have stopped this specific campaign, since no AEI was ever declared here. Their value is demonstrated only for a cooperative case we did not have an instance of.
- **Thresholds are illustrative, not calibrated.** B-INV-4’s 500-actions/day figure and B-INV-2’s churn-pattern check were chosen for plausibility. We did not have access to a legitimate dataset-processing traffic baseline to derive or validate either figure.
- **Single-incident derivation.** All four invariants, and the entire two-regime framework, were derived from one incident’s public record. We have not tested whether they generalize to a different kind of containment failure.
- **Verifies the platform’s view only.** Our design verifies what a third-party platform observed, not what happened inside the lab’s own infrastructure before an agent reached that platform. A lab could

contain nothing internally and rely entirely on downstream platforms to catch the fallout, which is closer to what happened here than is comfortable.

- **Detection-latency estimate is directional.** The one-to-three-hour figure in Section 4 is an order-of-magnitude estimate from daily totals, not measured hourly data.
- **Adoption of the preventive pair is not addressed by this work.** B-INV-1 and B-INV-3 only do useful work once labs declare AEIs when provisioning access to third-party infrastructure, and nothing in our design compels that declaration; it is a policy lever, not a technical one.

Dual-Use Considerations

Publishing exact behavioral thresholds hands a future attacker a map of exactly what stays under the radar. B-INV-4’s volume threshold and B-INV-2’s churn-pattern check are both published in this paper and in the reference monitor’s source code; an attacker aware of either could deliberately throttle request volume below the threshold, or vary session characteristics just enough to avoid matching a flagged identity’s behavioral signature, while still accomplishing the same underlying access. This risk is higher here than it would be for a standard kept internal to one lab, since these thresholds are meant to be adopted across many third-party platforms and not kept as one organization’s private detection logic.

A second, less obvious risk is specific to the two-regime design itself. Because B-INV-1 and B-INV-3 can only return `fail` or `pass` when an AI Evaluation Identity has been declared, a lab has a perverse incentive to simply never declare one: doing so guarantees both invariants report `not_applicable`. A careless reader or auditor could mistake a page of `not_applicable` verdicts for a clean record instead of for what it actually is, evidence that the checked party declined to participate in the half of the standard that requires cooperation. The standard’s own honesty depends on `not_applicable` being read as a distinct, and in context often worse, signal than `pass`, not as a neutral placeholder.

We mitigate the first risk by proposing, as in Section 5, that invariant categories be published and standardized while exact numeric thresholds remain private and platform-specific, calibrated against each platform’s own traffic baseline. We mitigate the second by recommending that any adoption of B-INV-1 and B-INV-3 report AEI-declaration rates alongside conformance verdicts, so that a pattern of non-declaration is itself visible and reportable, rather than silently absorbed into a string of `not_applicable` results.

LLM Usage Statement

Claude (Anthropic) was used for brainstorming, drafting the reference monitor and event trace, and editing the report. All factual claims, citations, and code outputs were independently checked against primary sources by the author before inclusion.